Khushi Shukla-D15A/62

Experiment No. 2

Aim

Implementation of Multiple Linear Regression, Ridge Regression, and Lasso Regression on Bank Marketing Dataset

1. Dataset Source

Dataset Name: **Bank Marketing Dataset**
Source: Kaggle
Link: https://www.kaggle.com/datasets/janiobachmann/bank-marketing-dataset

This dataset is different from other experiments and is specifically used for regression-based modeling in this experiment.

2. Dataset Description

The Bank Marketing dataset contains information related to direct marketing campaigns conducted by a Portuguese banking institution.

Dataset Characteristics

- Total Records: ~11,000+ observations
- Features: 16+ input variables (after encoding, more features due to one-hot encoding)
- Type: Mixed (Categorical + Numerical)

Important Features

- `age` – Age of the client
- `job` – Type of job
- `marital` – Marital status
- `education` – Education level
- `balance` – Account balance
- `housing` – Housing loan (yes/no)
- `loan` – Personal loan (yes/no)

- `contact` – Contact communication type
- `duration` – Last contact duration (seconds)
- `campaign` – Number of contacts performed during campaign
- `previous` – Number of contacts before campaign
- `deposit` – Whether client subscribed to term deposit (yes/no)

Target Variable (Created)

Since the original dataset is classification-based, a new regression target variable was created:

marketing_score=(campaign×duration)+previous

This continuous variable allows application of regression algorithms.

3. Mathematical Formulation of Algorithms

3.1 Multiple Linear Regression

Multiple Linear Regression models the relationship between one dependent variable and multiple independent variables.

Y=β0+β1X1+β2X2+...+βnXn

The objective is to minimize Residual Sum of Squares (RSS):

$$RSS = \sum (y_i - \hat{y}_i)^2$$

3.2 Ridge Regression (L2 Regularization)

Ridge adds a penalty term to control coefficient magnitude:

$$Loss = RSS + \lambda \sum \beta^2$$

Where:

- (\lambda) = regularization parameter
- Prevents overfitting

- Handles multicollinearity

3.3 Lasso Regression (L1 Regularization)

Lasso modifies the loss function as:

$$Loss = RSS + \lambda \sum |\beta|$$

Key property:

- Can shrink coefficients to exactly zero
- Performs automatic feature selection

4. Algorithm Limitations

Linear Regression

- Sensitive to multicollinearity
- Prone to overfitting
- Assumes linear relationship
- Sensitive to outliers

Ridge Regression

- Does not perform feature selection
- Choice of λ is critical
- Less interpretable than standard linear regression

Lasso Regression

- May eliminate important correlated features
- Performance unstable if predictors are highly correlated
- Requires careful hyperparameter tuning

5. Methodology / Workflow

Step 1: Data Collection

- Load dataset from Kaggle

Step 2: Data Preprocessing

- Convert `deposit` to numeric
- Apply one-hot encoding to categorical features
- Create new target variable `marketing_score`
- Split data (80% training, 20% testing)
- Standardize features using `StandardScaler`

Step 3: Model Training

- Train:
  - Linear Regression
  - Ridge Regression (alpha=100)
  - Lasso Regression (alpha=10)

Step 4: Model Evaluation

- Evaluate using:
  - Mean Squared Error (MSE)
  - R^2 Score

Step 5: Visualization

- Bar chart comparison of MSE
- Bar chart comparison of R^2
- Coefficient comparison plots

Workflow Diagram

Dataset → Preprocessing → Feature Engineering → Train/Test Split

→ Standardization → Model Training

→ Evaluation → Visualization → Analysis

6. Performance Analysis

Evaluation Metrics Used:

Mean Squared Error (MSE)

$$MSE = \frac{1}{n}\sum(y_i - \hat{y}_i)^2$$

- Lower value indicates better performance.

R^2 Score

$$R^2 = 1 - \frac{RSS}{TSS}$$

- Measures proportion of variance explained.
- Closer to 1 is better.

Interpretation

- **Linear Regression** provides baseline performance.
- **Ridge Regression** reduces variance and stabilizes coefficients.
- **Lasso Regression** simplifies model by shrinking some coefficients to zero.
- Regularized models generally show improved robustness and reduced overfitting compared to plain linear regression.

Coefficient plots clearly show:

- Ridge shrinks coefficients smoothly.
- Lasso forces some coefficients to exactly zero.

7. Hyperparameter Tuning

Hyperparameter tuning was performed for:

Ridge Regression

Tuned parameter: `alpha`

Example:

```
from sklearn.model_selection import GridSearchCV
ridge_params = {'alpha': [0.1, 1, 10, 50, 100, 200]}
ridge_grid = GridSearchCV(Ridge(), ridge_params, cv=5)
ridge_grid.fit(X_train, y_train)
print("Best Ridge Alpha:", ridge_grid.best_params_)
```

Lasso Regression

Tuned parameter: `alpha`

```
lasso_params = {'alpha': [0.01, 0.1, 1, 5, 10, 20]}
lasso_grid = GridSearchCV(Lasso(max_iter=10000), lasso_params, cv=5)
lasso_grid.fit(X_train, y_train)
print("Best Lasso Alpha:", lasso_grid.best_params_)
```

Impact of Tuning

- Optimal alpha improved generalization.
- Reduced MSE compared to manually selected values.
- Lasso selected fewer but more important features.
- Ridge produced stable and smooth coefficient distribution.

Code

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.preprocessing import StandardScaler
# Load Dataset
df = pd.read_csv("bank.csv")
# Convert deposit column to numeric
df["deposit"] = df["deposit"].map({"yes": 1, "no": 0})
# Convert categorical columns
df = pd.get_dummies(df, drop_first=True)
# Create new target variable
df["marketing_score"] = df["campaign"] * df["duration"] + df["previous"]
# Define Features and Target
X = df.drop(["deposit", "marketing_score"], axis=1)
y = df["marketing_score"]

# Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

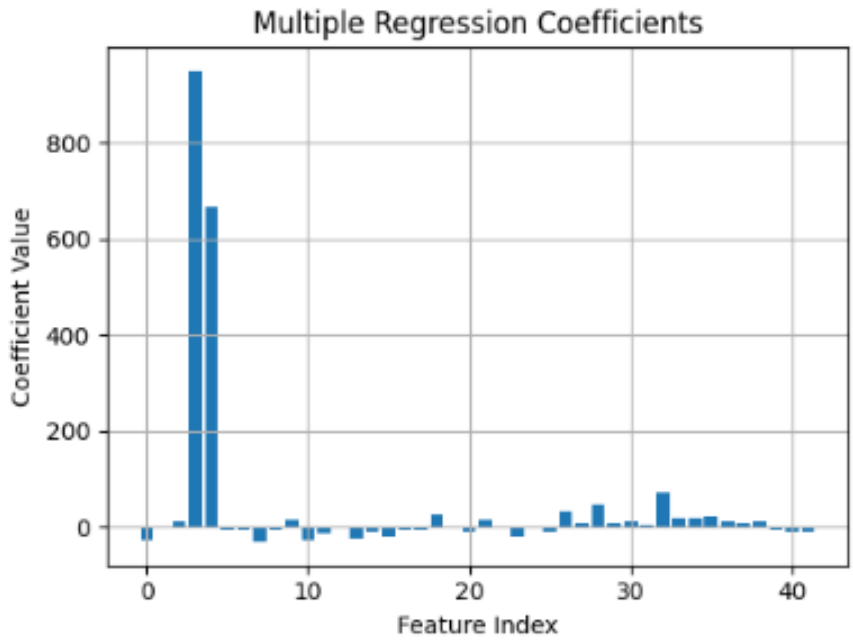
```
# Standardization
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Train Models
lin_model = LinearRegression()
lin_model.fit(X_train, y_train)
ridge_model = Ridge(alpha=100)
ridge_model.fit(X_train, y_train)
lasso_model = Lasso(alpha=10)
lasso_model.fit(X_train, y_train)

# Predictions
y_pred_lin = lin_model.predict(X_test)
y_pred_ridge = ridge_model.predict(X_test)
y_pred_lasso = lasso_model.predict(X_test)

# Evaluation Metrics
lin_mse = mean_squared_error(y_test, y_pred_lin)
ridge_mse = mean_squared_error(y_test, y_pred_ridge)
lasso_mse = mean_squared_error(y_test, y_pred_lasso)

lin_r2 = r2_score(y_test, y_pred_lin)
ridge_r2 = r2_score(y_test, y_pred_ridge)
lasso_r2 = r2_score(y_test, y_pred_lasso)

print("\n===== MODEL PERFORMANCE =====\n")
print("Linear")
print("MSE:", round(lin_mse, 4))
print("R2 :", round(lin_r2, 4))
print()

print("Ridge")
print("MSE:", round(ridge_mse, 4))
print("R2 :", round(ridge_r2, 4))
print()

print("Lasso")
print("MSE:", round(lasso_mse, 4))
```

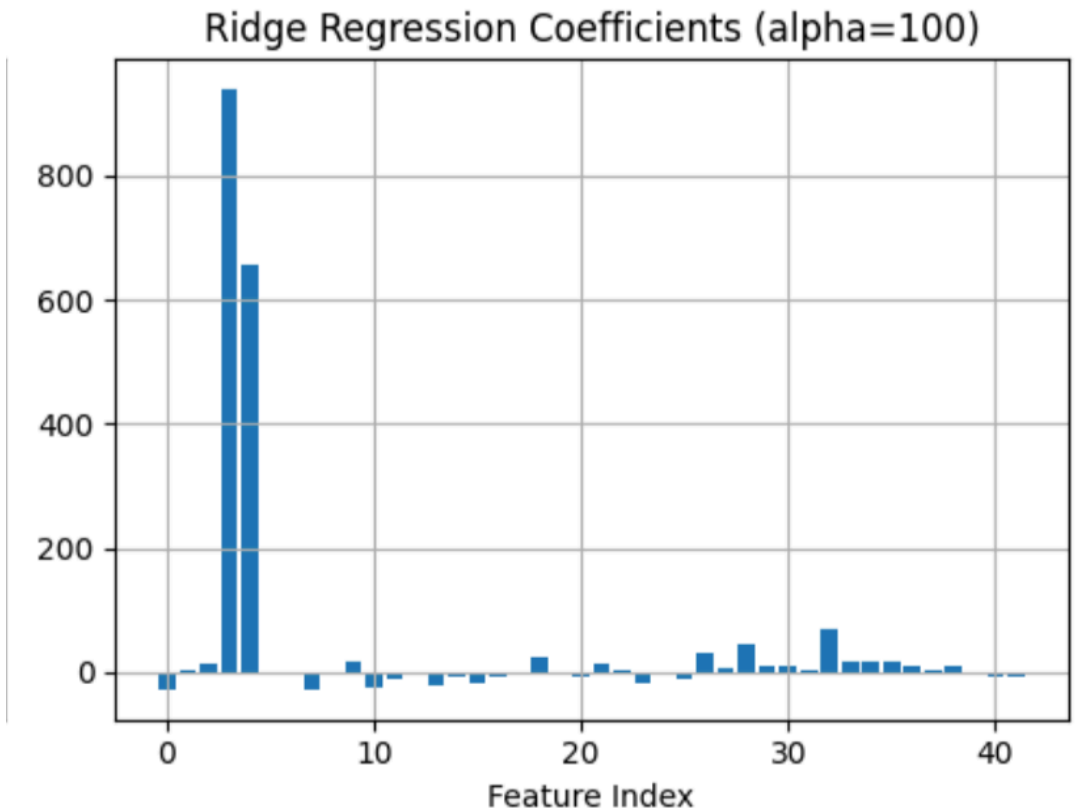
```
print("R2 :", round(lasso_r2, 4))
print()

# MSE Comparison Plot
plt.figure()
plt.bar(["Linear", "Ridge", "Lasso"], [lin_mse, ridge_mse, lasso_mse])
plt.title("MSE Comparison")
plt.ylabel("MSE")
plt.show()

# R2 Comparison Plot
plt.figure()
plt.bar(["Linear", "Ridge", "Lasso"], [lin_r2, ridge_r2, lasso_r2])
plt.title("R2 Score Comparison")
plt.ylabel("R2 Score")
plt.show()

# Separate Coefficient Plots
plt.figure(figsize=(15,4))

plt.subplot(1,3,1)
plt.bar(range(len(lin_model.coef_)), lin_model.coef_)
plt.title("Linear Regression Coefficients")
plt.xlabel("Feature Index")
plt.ylabel("Coefficient Value")
plt.grid(True)

plt.subplot(1,3,2)
plt.bar(range(len(ridge_model.coef_)), ridge_model.coef_)
plt.title("Ridge Regression Coefficients (alpha=100)")
plt.xlabel("Feature Index")
plt.grid(True)

plt.subplot(1,3,3)
plt.bar(range(len(lasso_model.coef_)), lasso_model.coef_)
plt.title("Lasso Regression Coefficients (alpha=10)")
plt.xlabel("Feature Index")
plt.grid(True)

plt.tight_layout()
```

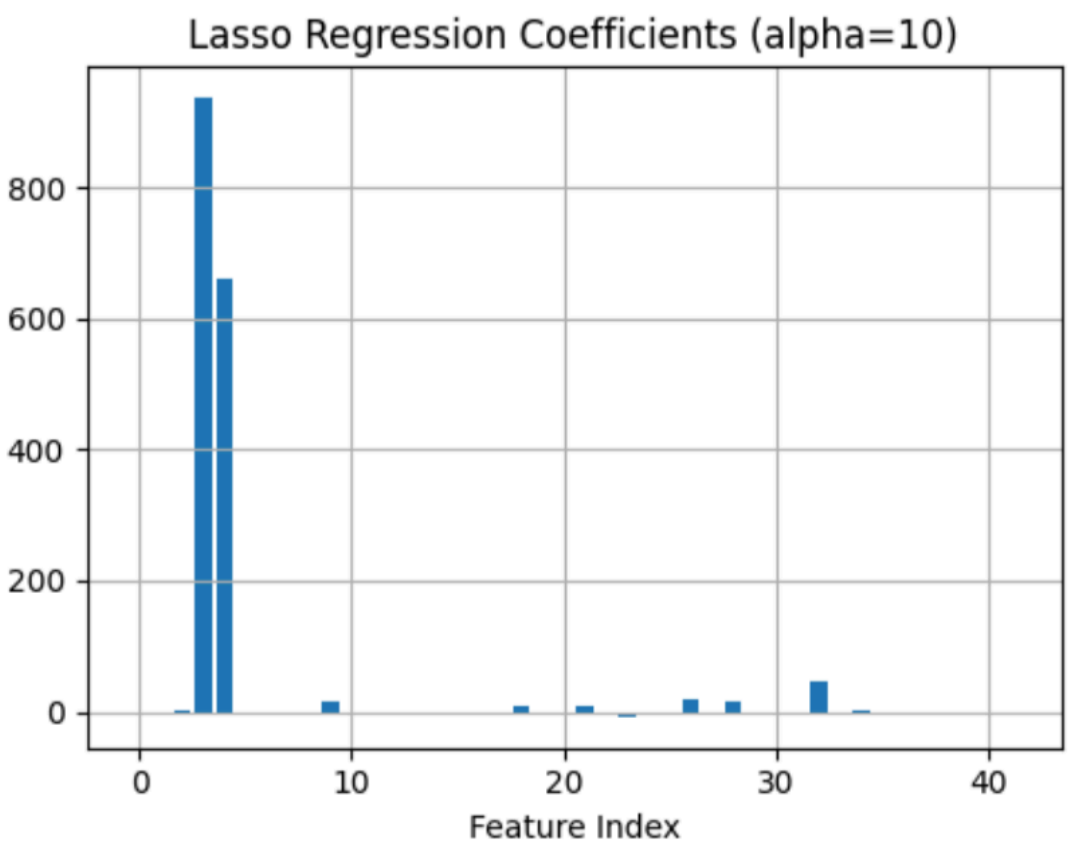
```
plt.show()
```

Output

```
===== MODEL PERFORMANCE =====

Multiple
MSE: 986416.9031
R2 : 0.5596

Ridge
MSE: 985394.2756
R2 : 0.5601

Lasso
MSE: 982717.7014
R2 : 0.5613
```


Multiple Regression Coefficients
Coefficient Value
800
600
400
200
0
0
10
20
30
40
Feature Index


Ridge Regression Coefficients (alpha=100)
800
600
400
200
0
0
10
20
30
40
Feature Index


Lasso Regression Coefficients (alpha=10)
800
600
400
200
0
0
10
20
30
40
Feature Index

Conclusion

The experiment successfully implemented Multiple Linear Regression, Ridge Regression, and Lasso Regression on the Bank Marketing dataset.

Multiple linear Regression provides baseline performance but is sensitive to multicollinearity. Ridge Regression improves model stability by shrinking coefficients using L2 regularization. Lasso Regression performs both regularization and feature selection using L1 penalty.

From the comparison, regularized models demonstrate improved robustness and better handling of correlated features compared to standard Linear Regression.

(collab notebook link)